

DEVELOPER GUIDE

Technical Engineering & Extension Manual — Vishal Gold

Project	Vishal Gold — B2B/B2C Jewelry App
Prepared For	Vishal Gold Engineering Team
Prepared By	Senior Backend & Mobile Architects
Version	v1.1
Date	February 25, 2026

Table of Contents

1. Architecture & philosophy
2. Project Structure Deep-Dive
3. Infrastructure: Firebase & Secure Storage
4. State Management (Provider Stack)
5. Administrative Staging Workflow
6. Security Architecture & VAPT Mitigation
7. Build & Environment Configuration
8. Developer Workflow & Standards

1. Architecture & Philosophy

The Vishal Gold application is built on a high-availability, security-first foundation. The system follows a reactive architecture where the UI is a function of the state, and the state is strictly managed through localized providers.

1.1 Reactive Data Flow

We utilize the 'Provider' package for state management. This ensures a clean separation between business logic and user interface. Every major domain (Auth, Cart, Products) lives in its own ChangeNotifier singleton.

- Unidirectional Data Flow: UI -> Provider Action -> Service -> Firestore.
- State Persistence: Critical PII is persisted in Secure Storage; catalog data in Firestore.
- Lazy Initialization: Providers are instantiated only when needed by the widget tree.

2. Project Structure Deep-Dive

The codebase is organized into atomic domains located in the lib/ folder.

Directory	Responsibility
config/	Firebase options and platform-specific configurations.
constants/	Design tokens (colors, text styles, spacing) and API strings.
models/	Strongly typed schemas with JSON serialization (Product, User, Order).
providers/	Business logic containers managing reactive state updates.
services/	Hardware/Backend interaction layer (Firebase, LocalStorage, Analytics).
screens/	Feature-specific UI modules (Admin, Wholesaler, Retailer flows).
widgets/	Re-usable design components (AppButtons, CustomCards, Shimmers).

3. Infrastructure: Firebase & Secure Storage

The backend is powered by Firebase, with a security layer implemented through hardware-backed storage.

3.1 FirebaseService

The `FirebaseService` (`lib/services/firebase_service.dart`) is the central API hub. It implements complex cloud logic including Staging, Batch Writes, and Audit Logs.

Module	Capability
Staging	Drafting changes in a separate collection for admin review.
Audit Logs	Tracking every admin write (Admin ID, Doc ID, Action).
Product API	Fetching filtered streams based on status (Draft/Published).
Order Engine	Atomic transaction handling for high-integrity orders.

3.2 SecureLocalStorageService

Unlike standard `SharedPreferences`, this service uses AES encryption via `flutter_secure_storage`. It is mandatory for storing any string that could identify a user (UID, Token, Role).

4. State Management (Provider Stack)

The `MultiProvider` setup in `main.dart` defines the dependency hierarchy of the application.

Provider	Primary Logic	Dependency
<code>AuthProvider</code>	OTP logic, Role verification, Profile sync.	None (Root)
<code>ProductProvider</code>	Catalog streaming, Category filtering.	<code>Auth</code> (Role-based price)
<code>CartProvider</code>	Local/Cloud basket sync, Item arithmetic.	<code>Auth</code> (UID matching)
<code>PreviewProvider</code>	Merging staged vs live data for admin view.	<code>Firebase</code> (Staging coll)

5. Administrative Staging Workflow

The app enforces a 'Standard Change' process to avoid catalog corruption.

Step 1: Admin edits product or banner details.

Step 2: Changes are written to the 'staging' collection (status=pending).

Step 3: Admin reviews the 'Staging Preview' screen (PreviewProvider).

Step 4: Admin clicks 'Publish All' — changes move to live collections.

6. Security Architecture & VAPT Mitigation

CRITICAL: The following security controls must never be removed or bypassed.

VAPT-001 (PII Security)

PII is strictly isolated in the SecureStorage provider. Raw SharedPreferences only stores UI flags.

VAPT-006 (Firestore Rules)

Minimum privilege rules: Wholesalers can only read /products/; Admins can write; Others are blocked.

VAPT-012 (Screenshot Protection)

The FLAG_SECURE window flag is toggled during initState on sensitive profile and cart screens.

7. Build & Environment Configuration

We use a Flavor-based setup (dev/prod) to separate Firebase projects. Release builds are optimized with R8 shrinking and obfuscation.

7.1 Build Commands

Standard Release APK: flutter build apk --release

Optimization: Build App Bundle (AAB) for Play Store deployment.

8. Developer Workflow & Standards

- PR Pre-check: Run 'flutter analyze' before any commit.
- Models: Always include 'toJson' and 'fromJson' for API compatibility.
- UI Consistency: Reference AppColors.gold instead of specific hex codes.
- Logging: Use debugPrint() for dev logs; wrapped in kDebugMode check.